

Read Mapping

Michael Schatz

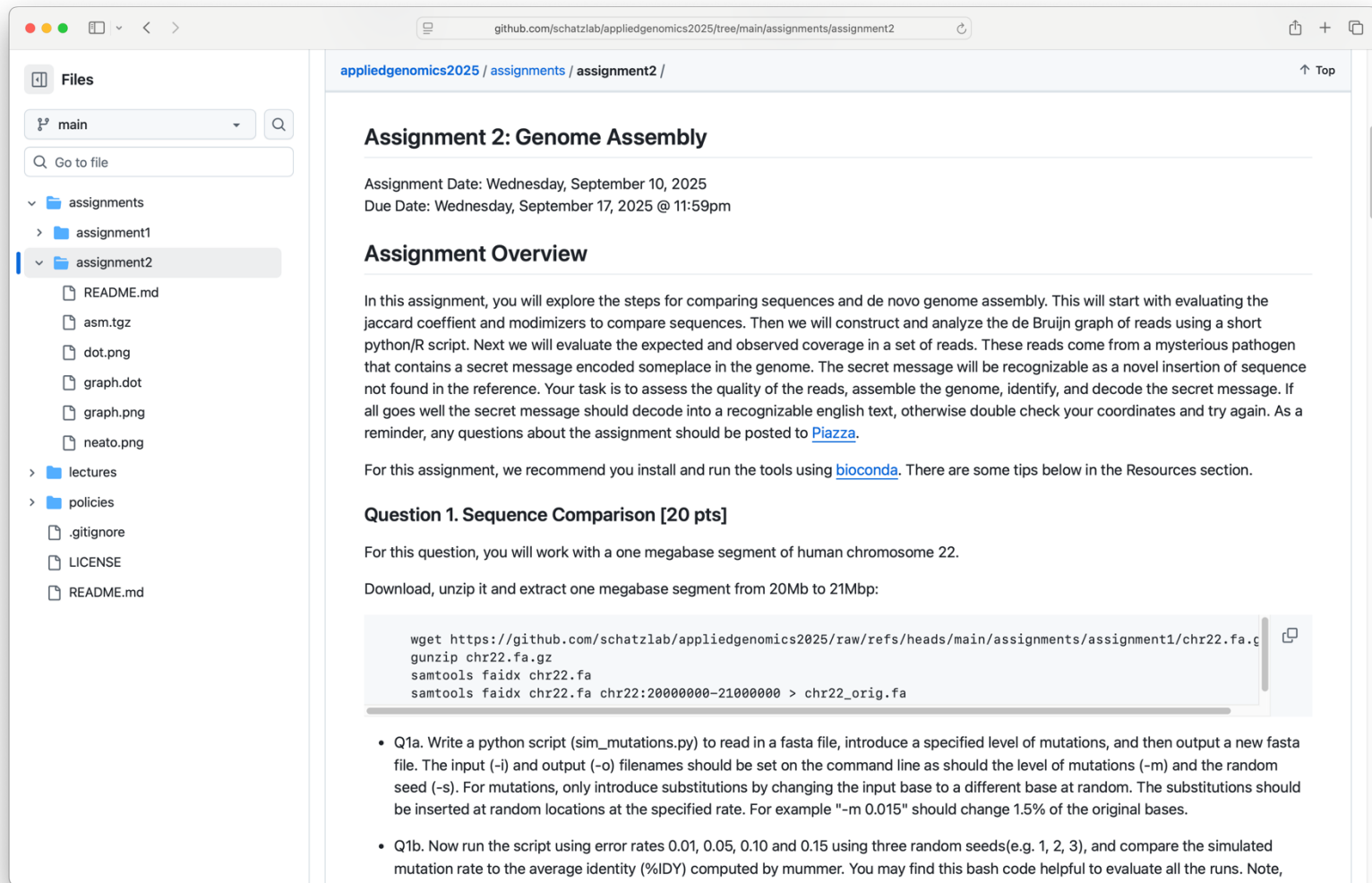
Sept 17, 2025

Lecture 7: Applied Comparative Genomics



Assignment 2: Genome Assembly

Due Wed Sept 17 by 11:59pm



The screenshot shows a web browser displaying the GitHub repository page for 'appliedgenomics2025/assignments/assignment2'. The left sidebar shows the file structure with folders 'assignments', 'assignment1', and 'assignment2'. The 'assignment2' folder is selected, showing files like 'README.md', 'asm.tgz', 'dot.png', 'graph.dot', 'graph.png', 'neato.png', 'lectures', 'policies', '.gitignore', 'LICENSE', and 'README.md'. The main content area is titled 'Assignment 2: Genome Assembly' and includes the assignment date (Wednesday, September 10, 2025) and due date (Wednesday, September 17, 2025 @ 11:59pm). Below this is an 'Assignment Overview' section explaining the task: comparing sequences and de novo genome assembly. It mentions evaluating the jaccard coefficient, modimizers, and constructing a de Bruijn graph. A secret message is hidden in the genome. The task is to assess the quality of the reads, assemble the genome, identify, and decode the secret message. A reminder to post questions on Piazza is included. Below the overview is 'Question 1. Sequence Comparison [20 pts]', which involves working with a one megabase segment of human chromosome 22. It provides instructions to download, unzip, and extract a segment from 20Mb to 21Mb. A code block shows the commands to download the file, unzip it, and extract the segment. Below the code block are two questions: Q1a. Write a python script (sim_mutations.py) to read in a fasta file, introduce a specified level of mutations, and then output a new fasta file. Q1b. Now run the script using error rates 0.01, 0.05, 0.10 and 0.15 using three random seeds (e.g. 1, 2, 3), and compare the simulated mutation rate to the average identity (%IDY) computed by mummer.

Files

main

Go to file

assignments

assignment1

assignment2

README.md

asm.tgz

dot.png

graph.dot

graph.png

neato.png

lectures

policies

.gitignore

LICENSE

README.md

appliedgenomics2025 / assignments / assignment2 /

Assignment 2: Genome Assembly

Assignment Date: Wednesday, September 10, 2025
Due Date: Wednesday, September 17, 2025 @ 11:59pm

Assignment Overview

In this assignment, you will explore the steps for comparing sequences and de novo genome assembly. This will start with evaluating the jaccard coefficient and modimizers to compare sequences. Then we will construct and analyze the de Bruijn graph of reads using a short python/R script. Next we will evaluate the expected and observed coverage in a set of reads. These reads come from a mysterious pathogen that contains a secret message encoded someplace in the genome. The secret message will be recognizable as a novel insertion of sequence not found in the reference. Your task is to assess the quality of the reads, assemble the genome, identify, and decode the secret message. If all goes well the secret message should decode into a recognizable english text, otherwise double check your coordinates and try again. As a reminder, any questions about the assignment should be posted to [Piazza](#).

For this assignment, we recommend you install and run the tools using [bioconda](#). There are some tips below in the Resources section.

Question 1. Sequence Comparison [20 pts]

For this question, you will work with a one megabase segment of human chromosome 22.

Download, unzip it and extract one megabase segment from 20Mb to 21Mb:

```
wget https://github.com/schatzlab/appliedgenomics2025/raw/refs/heads/main/assignments/assignment1/chr22.fa.gz
gunzip chr22.fa.gz
samtools faidx chr22.fa
samtools faidx chr22.fa chr22:20000000-21000000 > chr22_orig.fa
```

- Q1a. Write a python script (sim_mutations.py) to read in a fasta file, introduce a specified level of mutations, and then output a new fasta file. The input (-i) and output (-o) filenames should be set on the command line as should the level of mutations (-m) and the random seed (-s). For mutations, only introduce substitutions by changing the input base to a different base at random. The substitutions should be inserted at random locations at the specified rate. For example "-m 0.015" should change 1.5% of the original bases.
- Q1b. Now run the script using error rates 0.01, 0.05, 0.10 and 0.15 using three random seeds (e.g. 1, 2, 3), and compare the simulated mutation rate to the average identity (%IDY) computed by mummer. You may find this bash code helpful to evaluate all the runs. Note,

<https://github.com/schatzlab/appliedgenomics2025/tree/main/assignments/assignment2>

Check Piazza for questions!

Earth's heart of iron begins
to yield its secrets p. 18

Microglia in chronic pain recovery
and relapse pp. 33 & 86

Particle acceleration
in a nova explosion p. 77

Science

\$15
1 APRIL 2022
SPECIAL ISSUE
science.org



FILLING THE GAPS

Closing in on a complete
human genome p. 42

HOME > COLLECTIONS > COMPLETING THE HUMAN GENOME

COMPLETING THE HUMAN GENOME

A fully sequenced human genome was announced more than 20 years ago. However, owing to technological limitations, some genomic regions remained unresolved. Here, *Science* and other journals present research by the Telomere-to-Telomere (T2T) Consortium, reporting on the endeavor to complete a comprehensive human reference genome.

FILTERS

6 RESULTS FOUND

SPECIAL ISSUE RESEARCH ARTICLE

Segmental duplications and their variation in a complete human genome

BY MITCHELL R. VOLLGER, RAVI GUTART, PHILIP C. DISHACK, LUDOVICA MERCURI, WILLIAM T. HARVEY, ARIEL GERSHMAN, MARK DEKHAHI, ARVID SULOVARI, KATHERINE M. MUNDON, ALEXANDRA P. LEWIS, [...] EVAN E. EICHLE
SCIENCE • VOL. 376, NO. 6588 • 01 APR 2022



SPECIAL ISSUE RESEARCH ARTICLE

Complete genomic and epigenetic maps of human centromeres

BY NICOLAS ALTEMOSE, GLENNIS A. LOGSDON, ANDREY V. BZKADEZ, PRADYU BOHRAI, SASHA A. LANGLEY, DINA V. CALDAS, SAVANNAH J. HOYT, LEV URALSKY, FEDOR D. RYABOV, COLIN J. SHEW, [...] KAREN H. NIGRA
SCIENCE • VOL. 376, NO. 6588 • 01 APR 2022



SPECIAL ISSUE RESEARCH ARTICLE

From telomere to telomere: The transcriptional and epigenetic state of human repeat elements

BY SAVANNAH J. HOYT, JESSICA M. STORER, GABRIELLE A. HARTLEY, PATRICK G. S. GRADY, ARIEL GERSHMAN, LEONARDO G. DE LIMA, CHARLES LMOUSE, REZA HALABIAN, LUKE WOLINSKI, MATIAS RODRIGUEZ, [...] RACHEL A.
+16 authors • SCIENCE • VOL. 376, NO. 6588 • 01 APR 2022



SPECIAL ISSUE RESEARCH ARTICLE

A complete reference genome improves analysis of human genetic variation

BY SERGEY AGANEZOV, STEPHANIE M. YAN, DANIELA C. SOTO, MELANIE KIRSCH, SAMANTHA ZARATE, PAVEL AVDEYEV, DYLAN J. TAYLOR, KISHOR SHAFIN, ALANA SHUMATE, CHUNLIN XIAO, [...] MICHAEL C. SCHATZ
+22 authors • SCIENCE • VOL. 376, NO. 6588 • 01 APR 2022



SPECIAL ISSUE RESEARCH ARTICLE

Epigenetic patterns in a complete human genome

BY ARIEL GERSHMAN, MICHAEL C. S. SAURIA, RAVI GUTART, MITCHELL R. VOLLGER, PAUL W. HOOK, SAVANNAH J. HOYT, METEN JIAN, ALANA SHUMATE, ROHAM RAZAGHI, SERGEY KOREN, [...] WINSTON TAMP
+9 authors • VOL. 376, NO. 6588 • 01 APR 2022



SPECIAL ISSUE RESEARCH ARTICLE

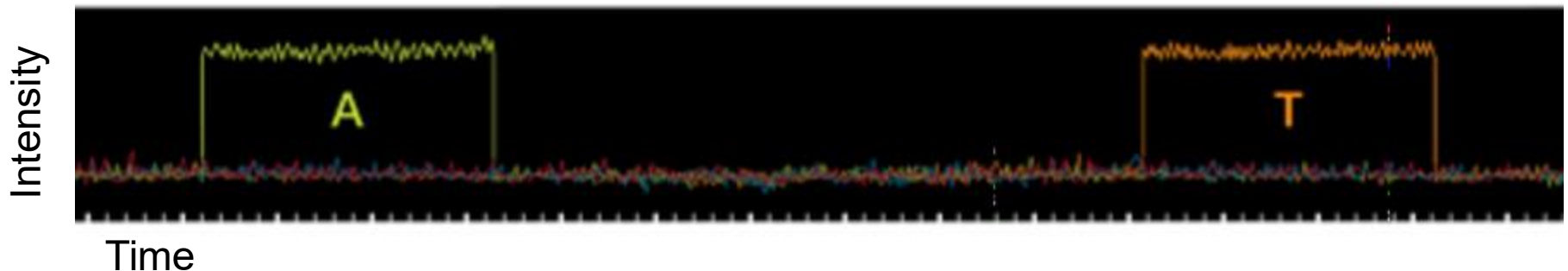
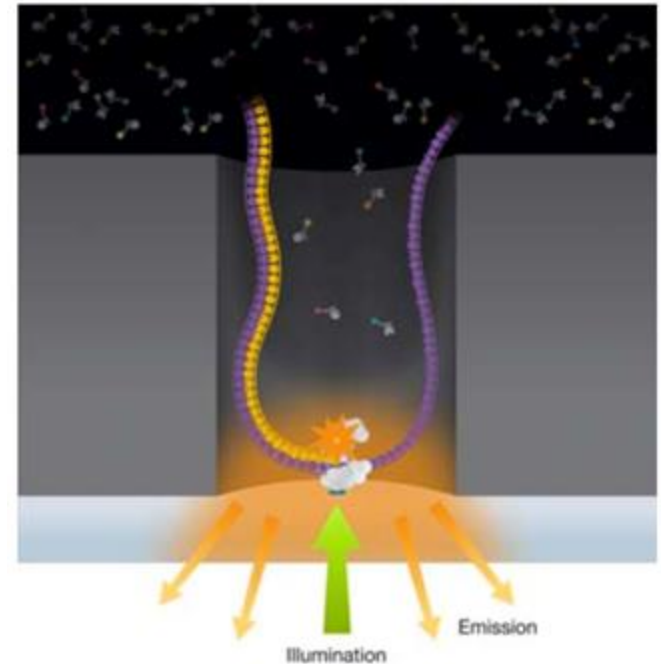
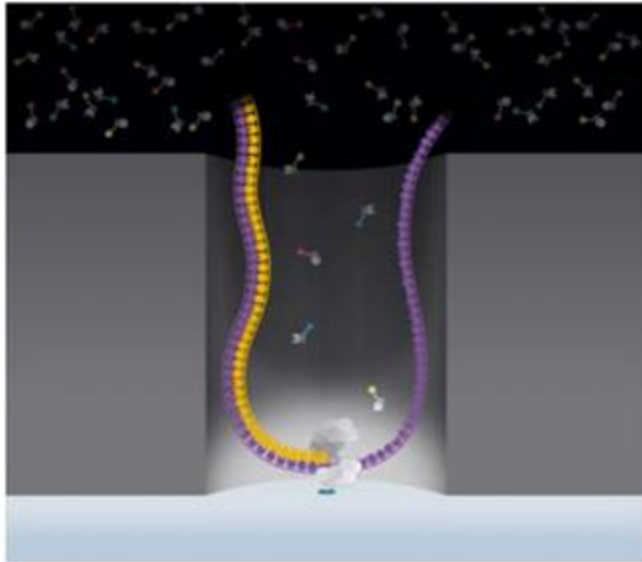
The complete sequence of a human genome

BY SERGEY KOREN, ARAND RHIE, MIKKO RAUTAIENEN, ANDREY V. BZKADEZ, ALLA MIKHENKO, MITCHELL R. VOLLGER, NICOLAS ALTEMOSE, LEV URALSKY, ARIEL GERSHMAN, [...] ADAM M. PHILLIPPY
+89 authors • VOL. 376, NO. 6588 • 01 MAR 2022



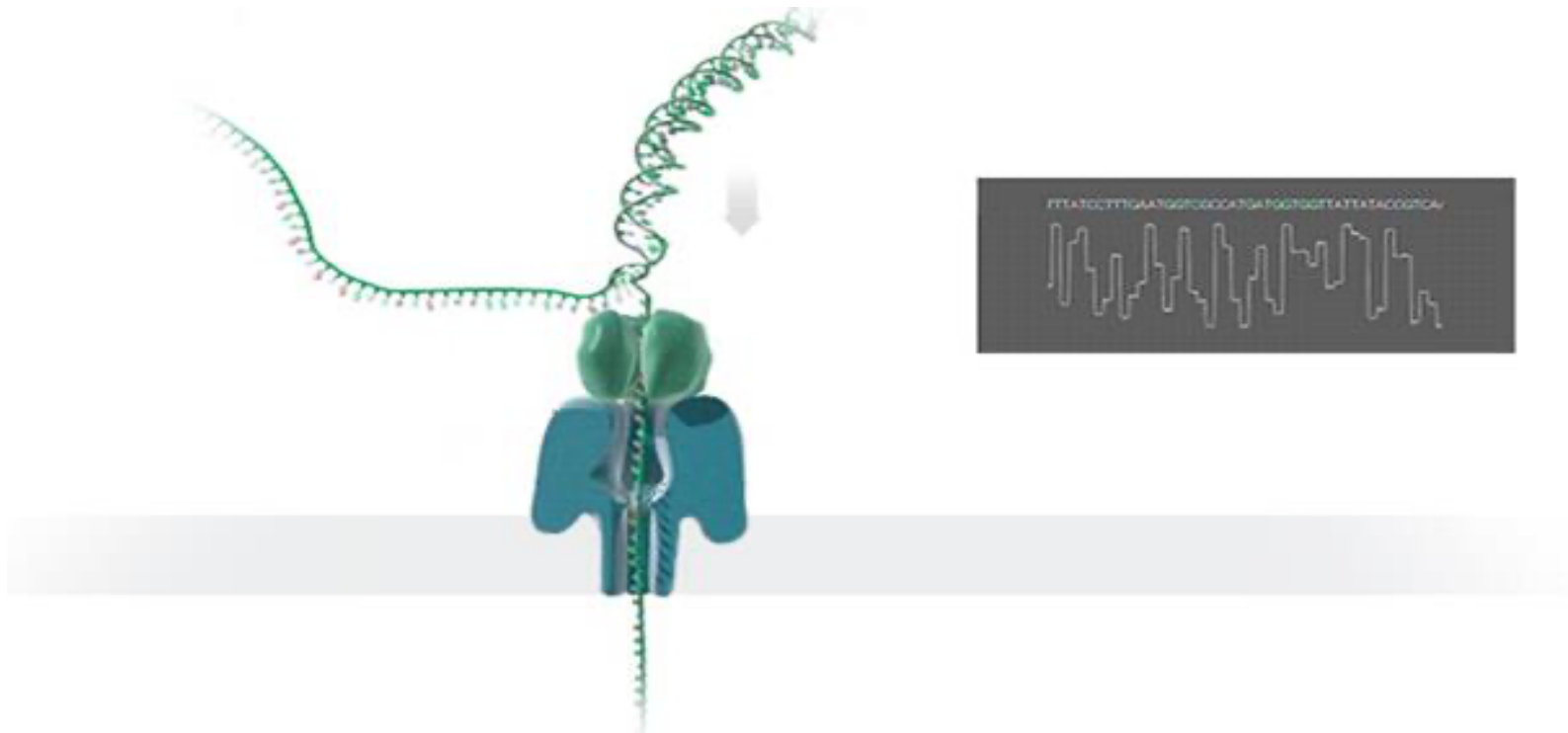
PacBio: SMRT Sequencing

Imaging of fluorescent phospholinked labeled nucleotides as they are incorporated by a polymerase anchored to a Zero-Mode Waveguide (ZMW).

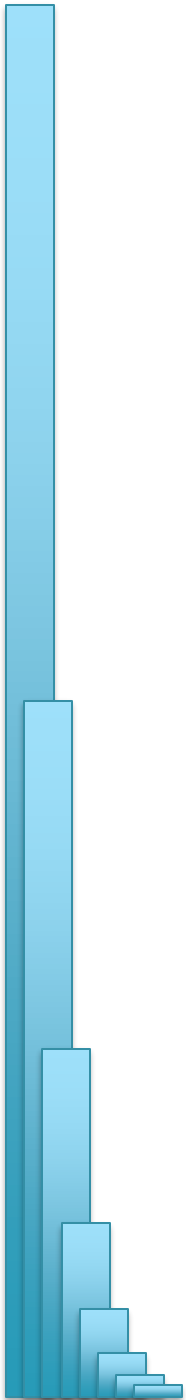


Nanopore Sequencing

Sequences DNA/RNA by measuring changes in ionic current as nucleotide strand passes through a pore

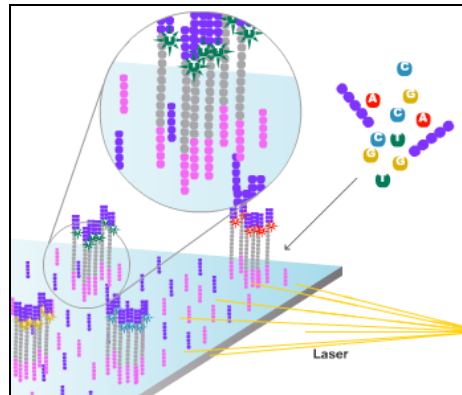
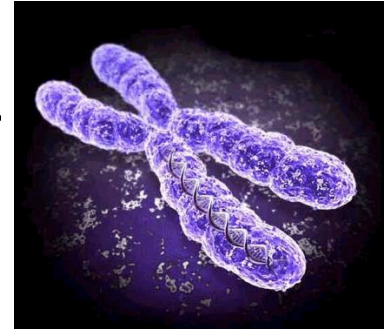
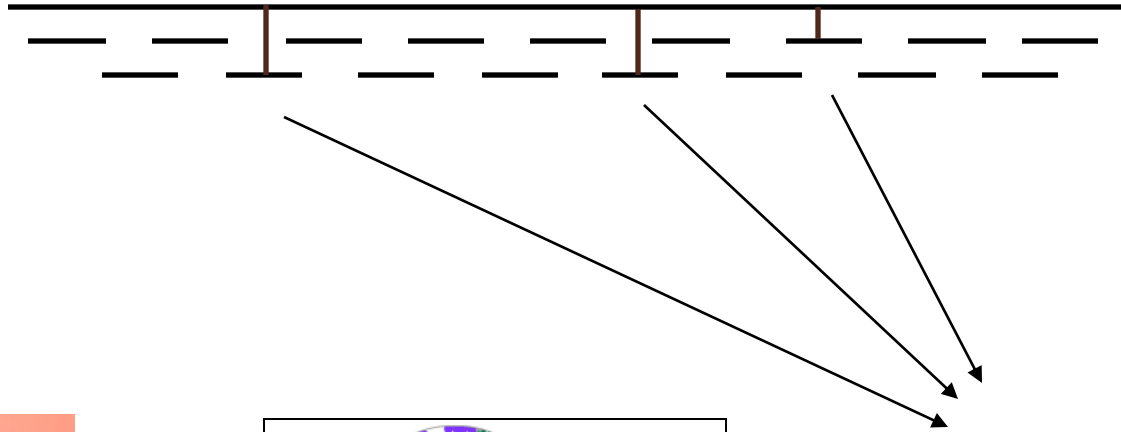


Read Mapping



Personal Genomics

How does your genome compare to the reference?



Heart Disease

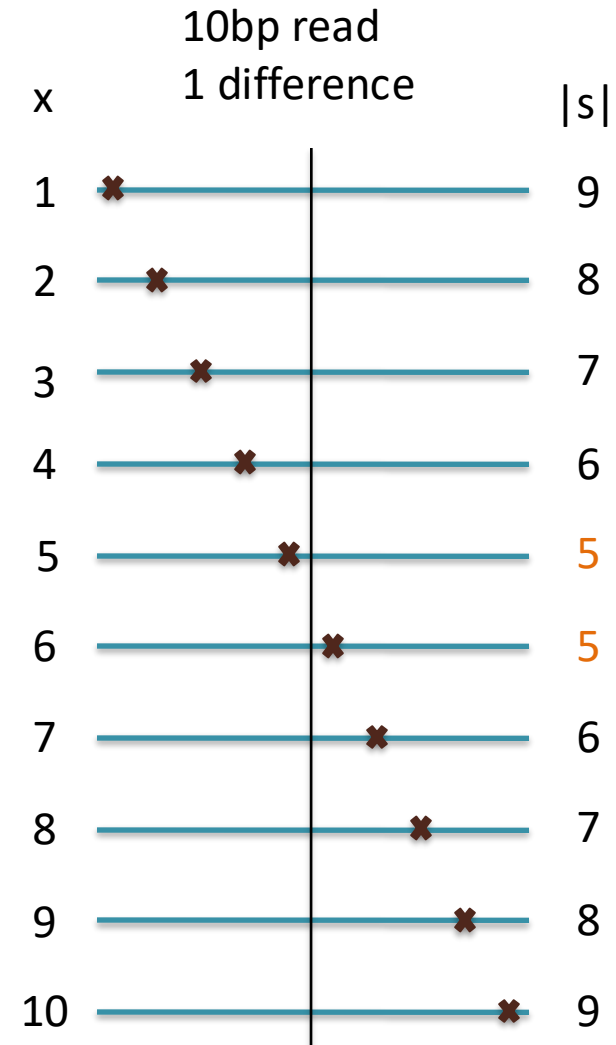
Cancer

Amazing actress

Seed-and-Extend Alignment

Theorem: An alignment of a sequence of length m with at most k differences **must** contain an exact match at least $s = m / (k + 1)$ bp long
(Baeza-Yates and Perleberg, 1996)

- Proof: Pigeonhole principle
 - 1 pigeon can't fill 2 holes
- Seed-and-extend search
 - Use an index to rapidly find short exact alignments to seed longer in-exact alignments
 - BLAST, MUMmer, Bowtie, BWA, SOAP, ...
 - Specificity of the depends on seed length
 - Guaranteed sensitivity for k differences
 - Also finds some (but not all) lower quality alignments <- heuristic



Brute Force Analysis



- Brute Force:
 - At every possible offset in the genome:
 - Do all of the characters of the query match?
- Analysis
 - Simple, easy to understand
 - Genome length = n [3B]
 - Query length = m [7]
 - Comparisons: $(n-m+1) * m$ [21B]
- Overall runtime: $O(nm)$
 - [How long would it take if we double the genome size, read length?]
 - [How long would it take if we double both?]

Brute Force Reflections

Why check every position?

- GATTACA can't possibly start at position 15

[WHY?]

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
T	G	A	T	T	A	C	A	G	A	T	T	A	C	C	...
								G	A	T	T	A	C	A	

- Improve runtime to $O(n + m)$

[3B + 7]

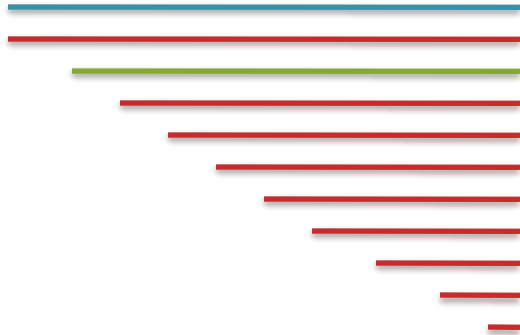
- If we double both, it just takes twice as long
- Knuth-Morris-Pratt, 1977
- Boyer-Moyer, 1977, 1991

- For one-off scans, this is the best we can do (optimal performance)

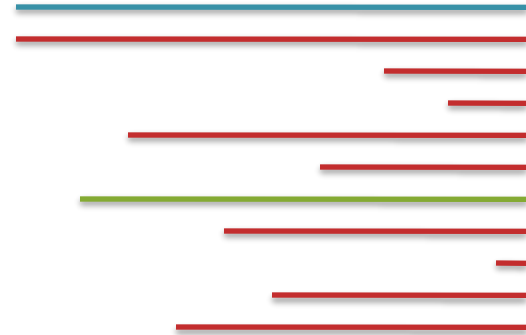
- We have to read every character of the genome, and every character of the query
- For short queries, runtime is dominated by the length of the genome

Suffix Arrays: Searching the Phone Book

- What if we need to check many queries?
 - We don't need to check every page of the phone book to find 'Schatz'
 - Sorting alphabetically lets us immediately skip 96% (25/26) of the book *without any loss in accuracy*
- Sorting the genome: Suffix Array (Manber & Myers, 1991)
 - Sort every suffix of the genome



Split into n suffixes



Sort suffixes alphabetically

[Challenge Question: How else could we split the genome?]

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - Lo = 1; Hi = 15;

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$

Lo
→

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
→

Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 9;$

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
Hi
→

Searching the Index

- Strategy 2: Binary search
 - Compare to the middle, refine as higher or lower
- Searching for GATTACA
 - $Lo = 1; Hi = 15; Mid = (1+15)/2 = 8$
 - $Middle = Suffix[8] = CC$
=> Higher: $Lo = Mid + 1$
 - $Lo = 9; Hi = 15; Mid = (9+15)/2 = 12$
 - $Middle = Suffix[12] = TACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 11; Mid = (9+11)/2 = 10$
 - $Middle = Suffix[10] = GATTACC$
=> Lower: $Hi = Mid - 1$
 - $Lo = 9; Hi = 9; Mid = (9+9)/2 = 9$
 - $Middle = Suffix[9] = GATTACA...$
=> Match at position 2!

#	Sequence	Pos
1	ACAGATTACC...	6
2	ACC...	13
3	AGATTACC...	8
4	ATTACAGATTACC...	3
5	ATTACC...	10
6	C...	15
7	CAGATTACC...	7
8	CC...	14
9	GATTACAGATTACC...	2
10	GATTACC...	9
11	TACAGATTACC...	5
12	TACC...	12
13	TGATTACAGATTACC...	1
14	TTACAGATTACC...	4
15	TTACC...	11

Lo
Hi
→

Binary Search Analysis

- Binary Search

Initialize search range to entire list

$\text{mid} = (\text{hi} + \text{lo}) / 2$; $\text{middle} = \text{suffix}[\text{mid}]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n / (2^x) \leq 1$; $x = \lg_2(n)$

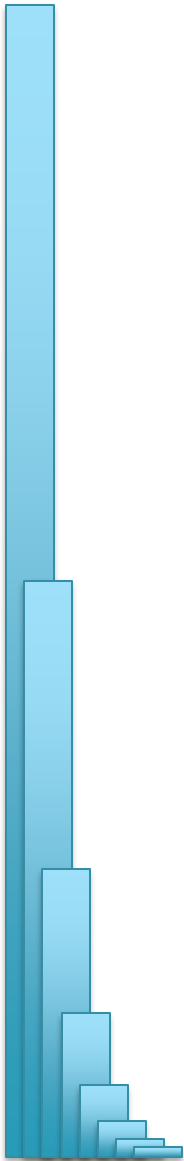
[32]

- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

- Looking up a query loops 32 times instead of 3B

[How long does it take to search 6B or 24B nucleotides?]



Binary Search Analysis

- Binary Search

Initialize search range to entire list

$\text{mid} = (\text{hi} + \text{lo}) / 2$; $\text{middle} = \text{suffix}[\text{mid}]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n / (2^x) \leq 1$; $x = \lg_2(n)$

[32]

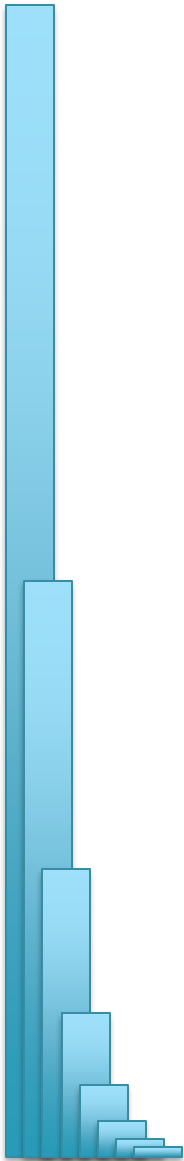
- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

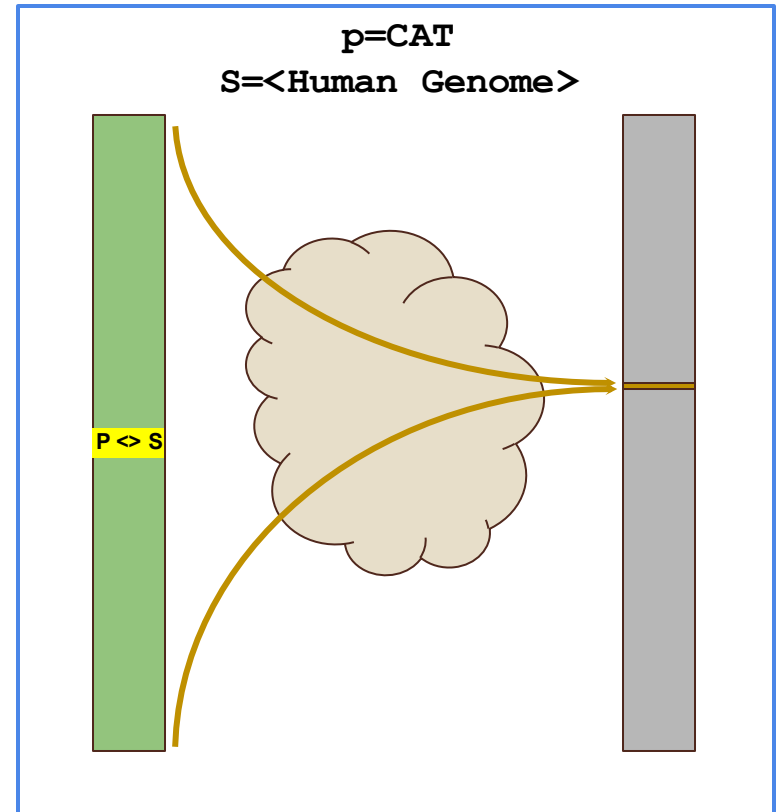
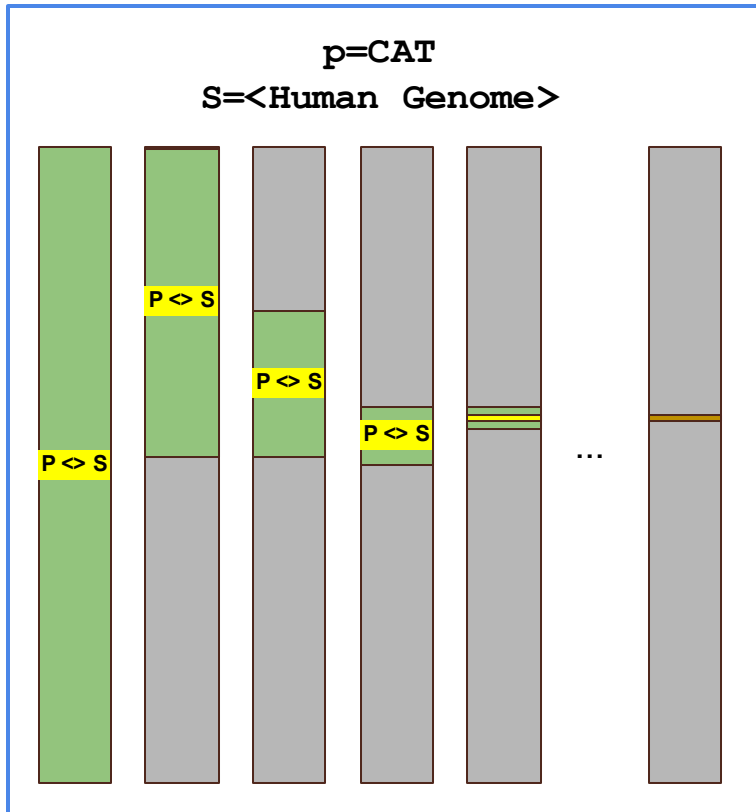
- Looking up a query loops 32 times instead of 3B

Can be reduced to $O(m + \lg n)$

using an auxiliary data structure called the LCP array



Sapling: Accelerating Suffix Array Queries with Learned Data Models



What if instead of a slow algorithmic approach to find the correct rows, we could somehow quickly guess/predict the correct rows?

Kirsche, M, Das, A, Schatz, MC (2020) Bioinformatics
doi: <https://doi.org/10.1093/bioinformatics/btaa911>

Algorithmic challenge

How can we combine the speed of a suffix array $O(m + \lg(n))$ (or even $O(m)$) with the size of a brute force analysis (n bytes)?

What would such an index look like?



Bowtie: Ultrafast and memory efficient alignment of short DNA sequences to the human genome

Slides Courtesy of Ben Langmead

Algorithm Overview

1. Split read into segments

Read

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA

Read (reverse complement)

TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: TCAGCCTTATTTACC

+, 20: **TTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: **TCAGCCTTATTTACC**

+ , 20: **TTTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: **GGTAAAATAAGGCTGA**

-, 20: GGCTGAGAGCTACTGG

Ungapped
alignment with
FM Index

Seed alignments (as B ranges)

$$\{ [211, 212], [212, 214] \}$$
$$\{ [653, 654], [651, 653] \}$$

{ [684, 685] }

{ }

{ }

{ [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955

SA:624, chr2:462

SA:211: chr4:762

SA:213: chr12:1935

SA:652: chr12:1945

SIMD dynamic
programming
aligner

SAM alignments

```

r1      0      chr12      1936      0
      36M      *      0      0
      CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
      IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
      AS:i:0      XS:i:-2      XN:i:0
      XM:i:0      XO:i:0      XG:i:0
      NM:i:0      MD:Z:36      YT:Z:UU
      YM:i:0

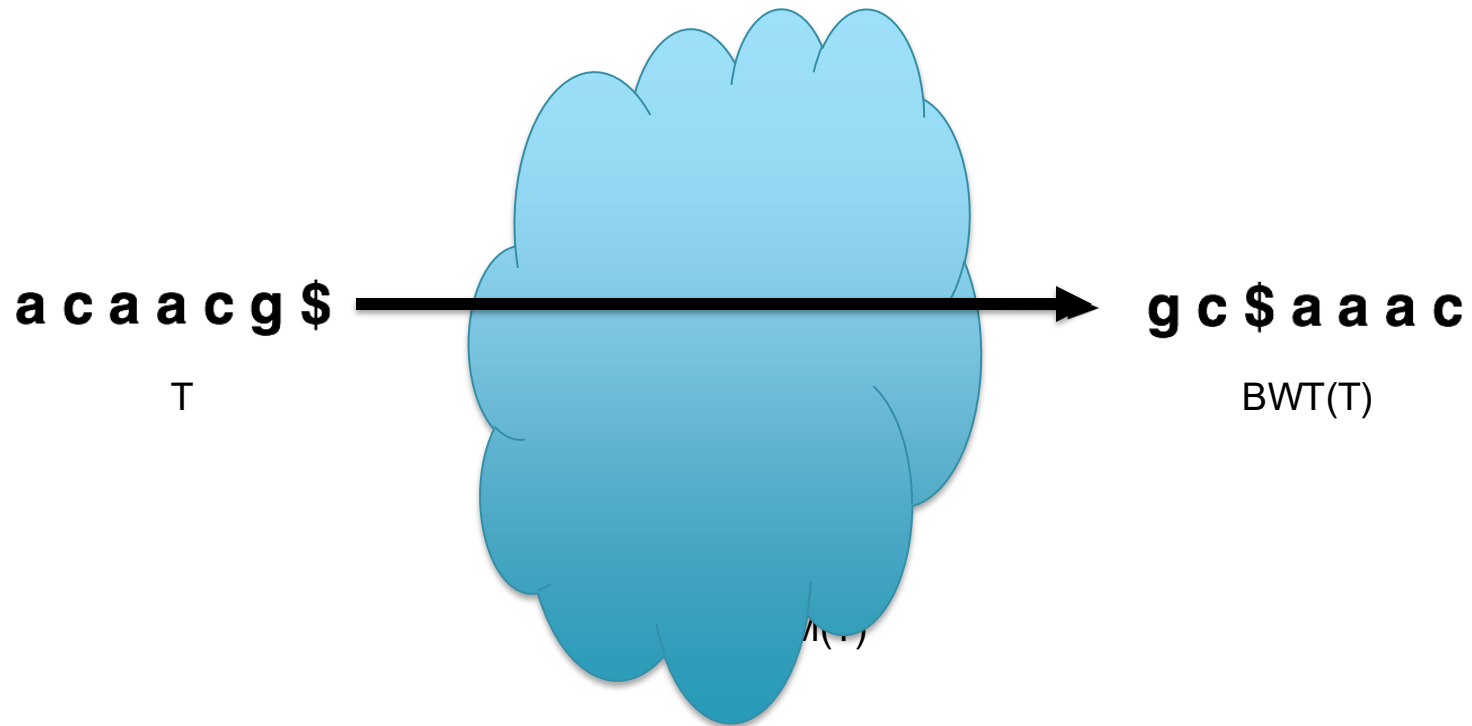
```

..

(Langmead & Salzberg, 2012)

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text

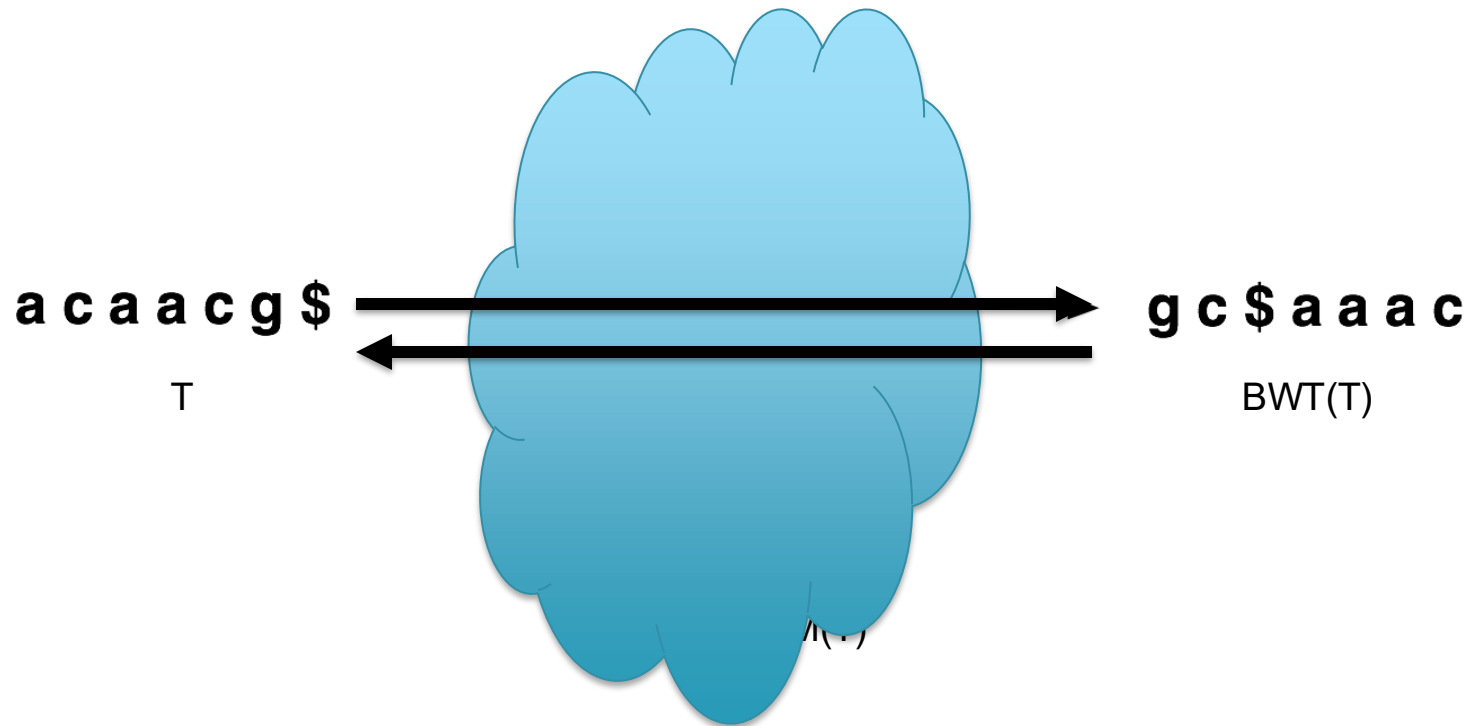


A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text



A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text

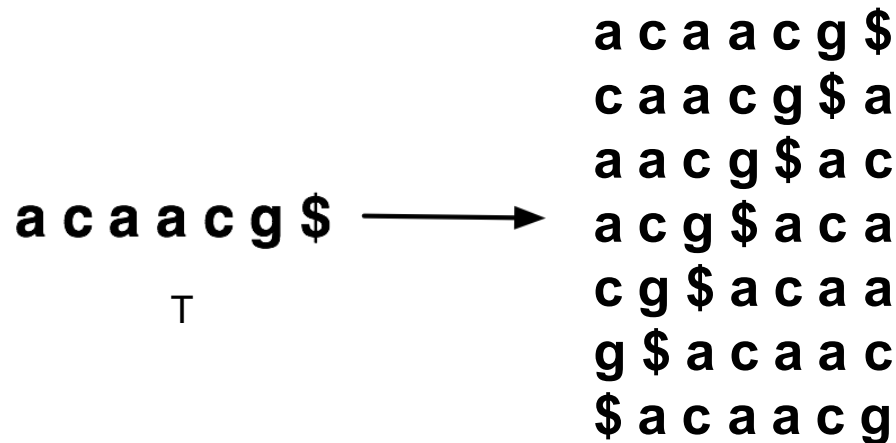
a c a a c g \$ →
T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



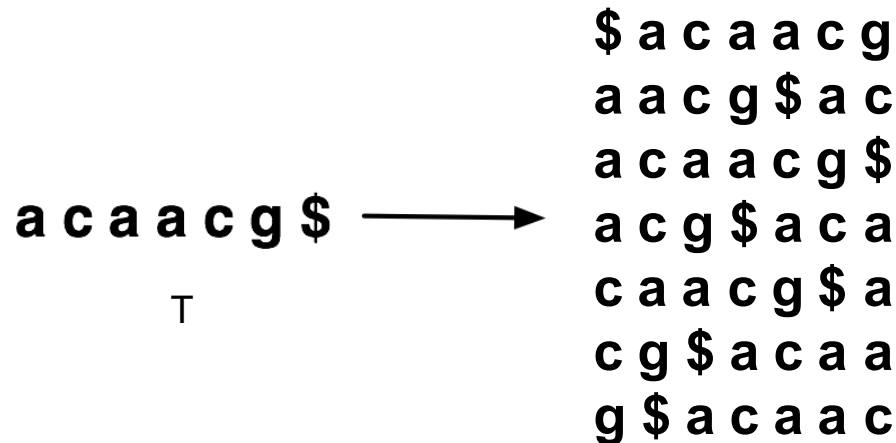
All cyclic permutations

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



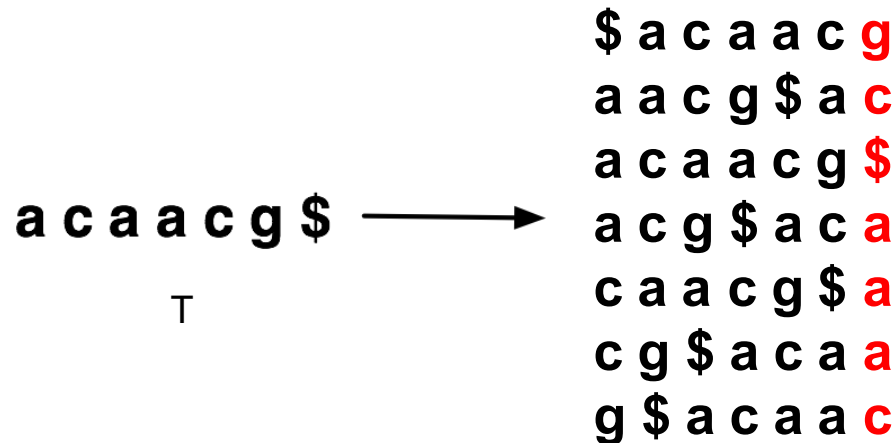
Sorted cyclic permutations
AKA Burrows Wheeler Matrix

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



Sorted cyclic permutations
AKA Burrows Wheeler Matrix

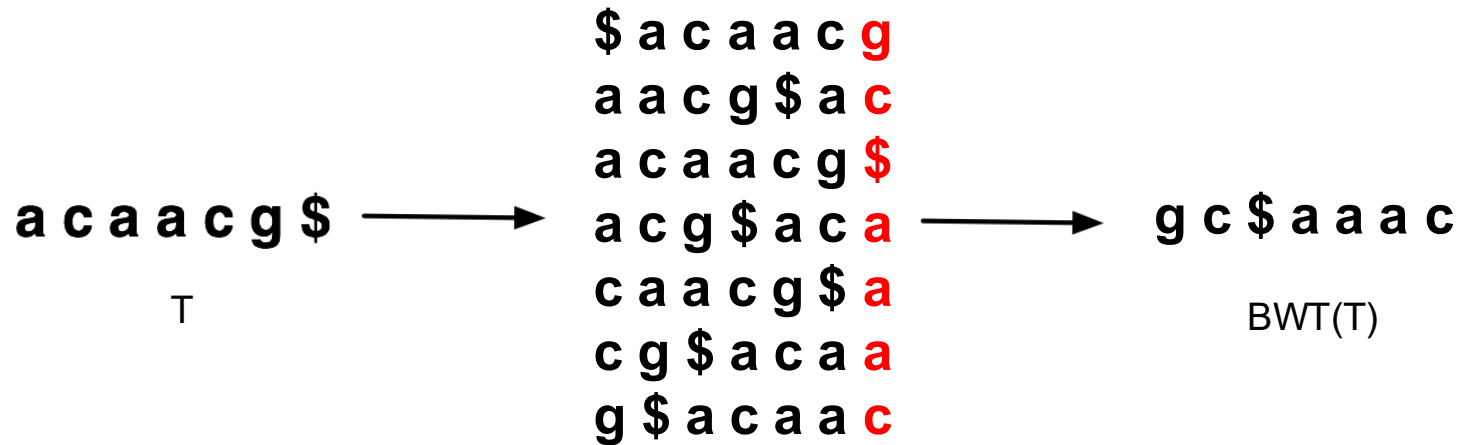
Last Column = Burrows Wheeler Transform

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



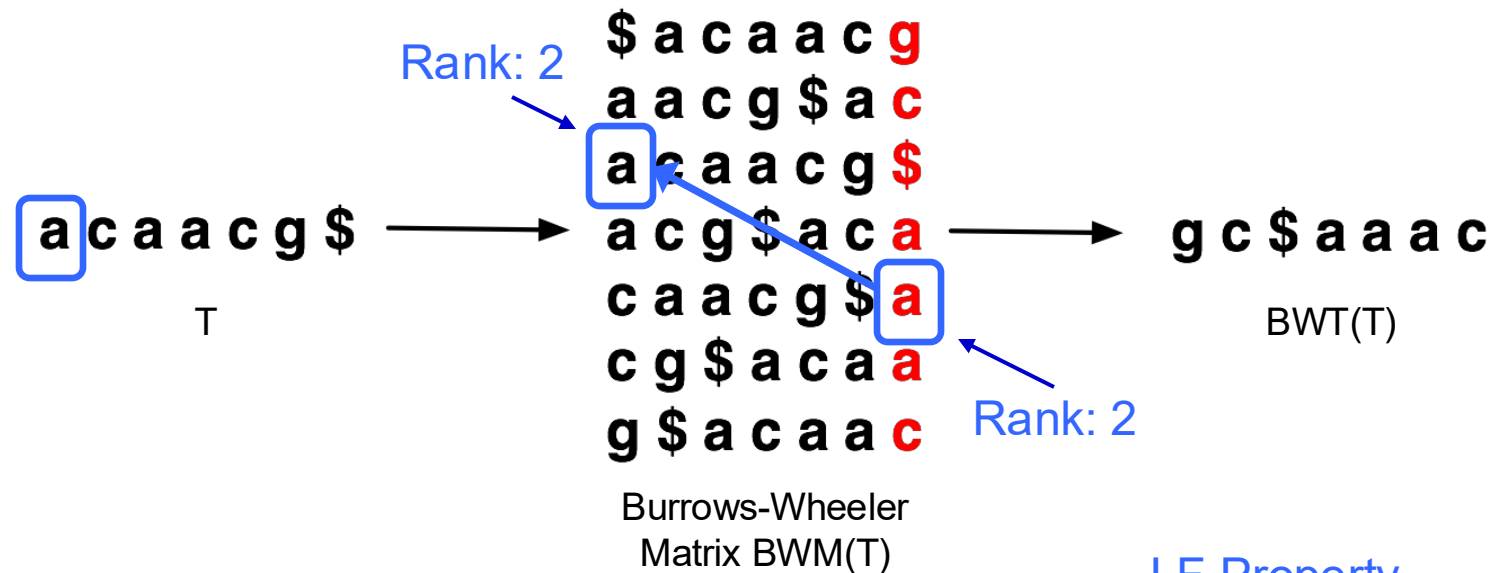
BWT(T) is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text

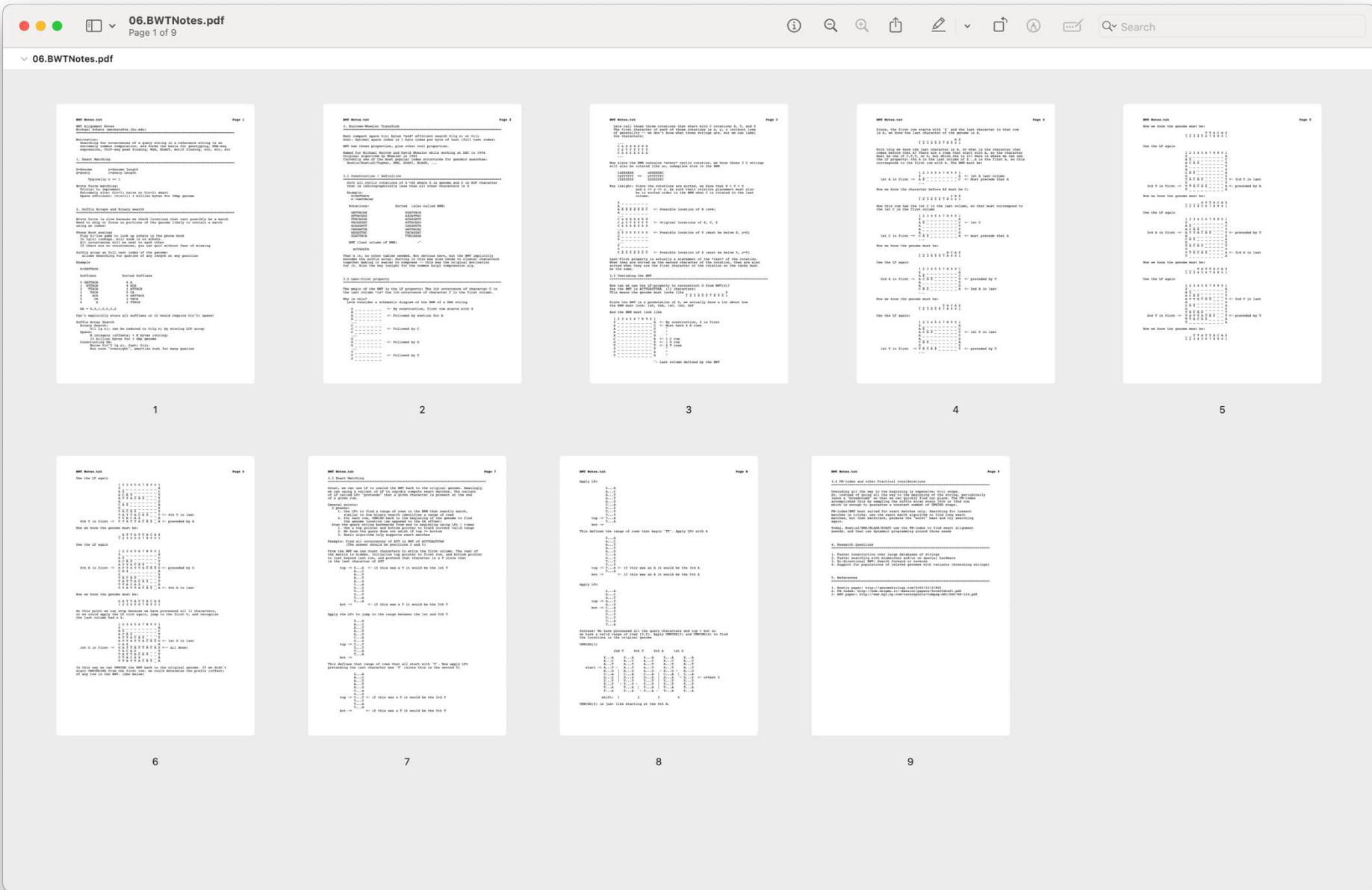


LF Property
implicitly encodes
Suffix Array

- $BWT(T)$ is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124



Recompute the original text when the BWT is ACTTGA\$TTAA

Burrows-Wheeler Transform

- Recreating T from BWT(T)
 - Start in the first row and apply **LF** repeatedly, accumulating predecessors along the way



[Decode this BWT string: ACTGA\$TTA]